

ACME Enterprise Systems

Advanced Engineering & Systems Architecture

28. Generative AI Architecture: Graph vs. Vector Trade-offs

Document ID: ENG-AI-092 | Author: AI Platform Engineering | Last Updated: June 2026

28.1 Hybrid Retrieval Strategy

ACME's core reasoning engine employs a hybrid retrieval strategy, acknowledging the technical trade-offs between pure vector-based RAG and structured Knowledge Graphs. While standard RAG excels at retrieving unstructured documentation using cosine similarity, it often hallucinates on multi-hop logical deductions.

To resolve this, entity relationships (e.g., User A belongs to Tenant B which holds License C) are stored as nodes and edges within our Knowledge Graph. When a query is initiated, the prompt router evaluates the structural complexity of the request.

28.2 SDK Implementation and Latency Routing

We maintain parallel integrations using both the Groq SDK and the OpenAI SDK to optimize the cost-to-latency ratio.

- **Groq Integration:** Utilized strictly for the initial prompt routing layer and real-time UI chatbots. The Groq LPU inference engine provides near-instantaneous generation, minimizing Time To First Token (TTFT).
- **OpenAI Integration:** Reserved for deep semantic analysis, data summarization, and extracting Cypher queries to execute against the Knowledge Graph.

29. Frontend Monorepo & UI Architecture Standards

Document ID: ENG-FE-101 | Author: Frontend Core Team | Last Updated: May 2026

29.1 Turborepo Structure & TypeScript Enforcement

The ACME frontend ecosystem operates within a strict Turborepo architecture to maximize build caching. All shared UI components and utility functions are isolated into local packages within the monorepo workspace. TypeScript is mandated across all applications with the compiler option `strict: true` enforced at the root `tsconfig.json`.

29.2 Tailwind CSS Configuration & Global Overrides

To prevent CSS class collisions in our Next.js App Router environments, all styling is governed by a centralized Tailwind configuration.

- **Tailwind Config:** The base `tailwind.config.ts` is exported from the `@acme/ui` package and extended by individual web applications. Custom brand colors and typography scales must be defined here rather than hardcoded in utility classes.
- **Global Styles:** Modifications to the `global.css` file are heavily restricted. Standard DOM resets are permitted, but component-level styling must remain within the utility-class paradigm or utilizing CSS modules for complex, state-driven animations.

30. Authentication State & ORM Synchronization

Document ID: ENG-AUTH-210 | Author: Identity & Access Team | Last Updated: April 2026

30.1 NextAuth.js Session Callbacks

Session state management relies on NextAuth.js configured with a custom Prisma adapter. To ensure the JWT contains the necessary tenant context for the Row-Level Security (RLS) policies defined in Document 25, the NextAuth session callback is modified to inject the active `tenant_id` into the token payload.

30.2 Prisma Schema & Dockerized Environments

Database migrations and client generation must remain synchronized across all developer machines and CI/CD pipelines.

- **Schema Management:** All database models and enums are defined within a single `schema.prisma` file. Pull requests modifying this schema must include the auto-generated migration SQL files.

- **Local Development:** Engineers must utilize the standard `docker-compose.yml` file to spin up isolated PostgreSQL and Redis instances. The application containers are configured to automatically execute `npx prisma migrate dev` upon startup, ensuring the local database schema matches the active branch.